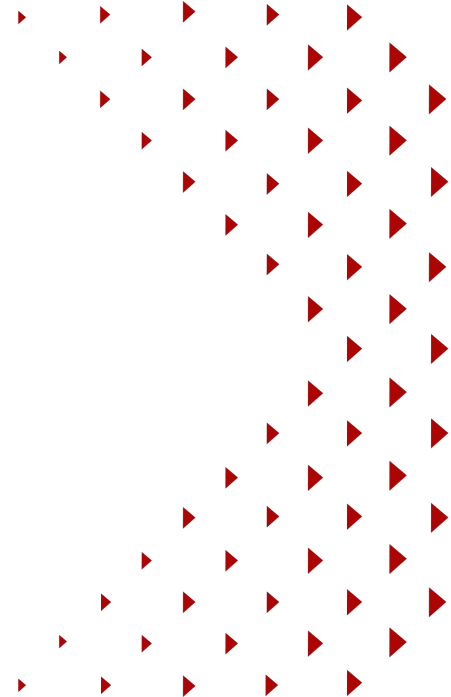




Session 03:

Conditions & Loops

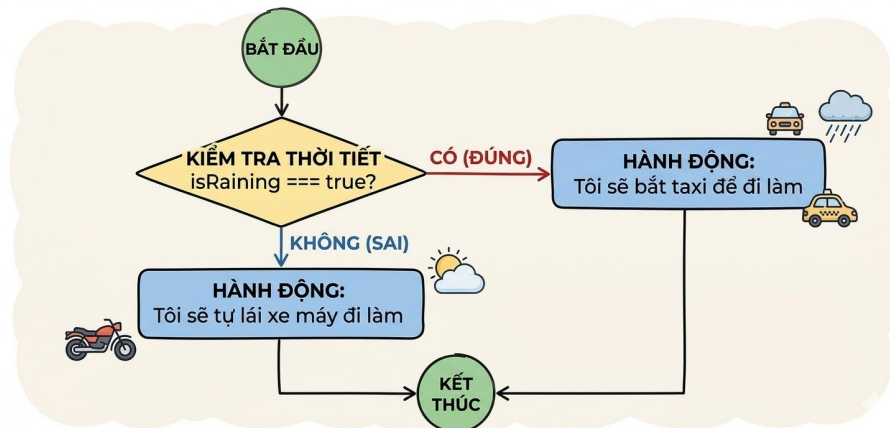


1. Biết, hiểu được về Cấu trúc điều kiện
2. Hiểu, áp dụng được Vòng lặp trong JavaScript
3. Sử dụng được biến điều khiển vòng lặp (Break, Continue)

1. Cấu trúc điều kiện

Conditional Statements

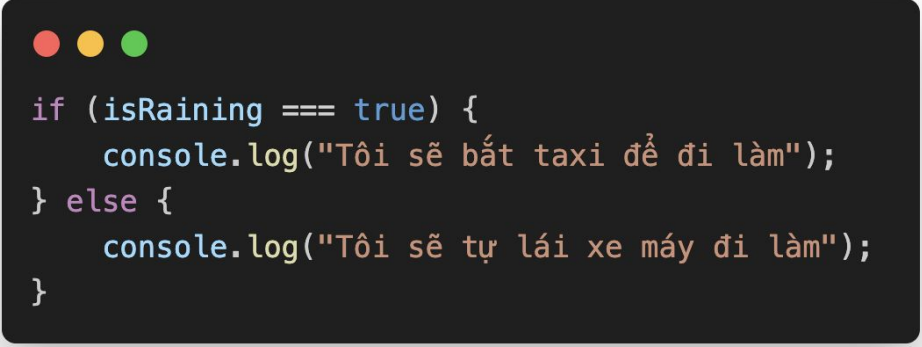
- Câu lệnh có điều kiện cho phép chúng ta thực hiện các hành động khác nhau cho các điều kiện khác nhau
- Câu lệnh có điều kiện chạy mã khác nhau tùy thuộc vào điều kiện đúng hay sai



Cú pháp cấu trúc điều kiện

Cú pháp

```
if (<condition>) {  
    // Mã được thực thi khi điều kiện đúng  
}  
else {  
    // Mã được thực thi khi điều kiện sai  
}
```



```
if (isRaining === true) {  
    console.log("Tôi sẽ bắt taxi để đi làm");  
} else {  
    console.log("Tôi sẽ tự lái xe máy đi làm");  
}
```

Câu lệnh if else if

Cú pháp

```
if (<condition1>) {  
    // Mã được thực thi khi condition1 đúng  
} else if (<condition2>) {  
    // Mã được thực thi khi condition2 đúng  
else {  
    // Mã được thực thi khi condition1 và  
    condition2 sai  
}
```

```
let rainLevel = "mưa to";  
// Các giá trị có thể: "không mưa", "mưa lâm râm", "mưa to"  
  
if (rainLevel === "không mưa") {  
    console.log("Thời tiết đẹp, xách xe máy đi làm thôi!");  
} else if (rainLevel === "mưa lâm râm") {  
    console.log("Mưa nhỏ thôi, chỉ cần mặc áo mưa mỏng là đủ.");  
} else {  
    // Trường hợp còn lại là mưa to  
    console.log(  
        "Mưa to quá! Nên bắt Grab car để không bị ướt hoặc đợi tạnh rồi đi.");  
}
```

Điều kiện lồng nhau

```
let isRaining = true;
let hasCrushEscort = true; // Có người thương đưa đón

if (isRaining) {
  if (hasCrushEscort) {
    console.log(
      "Mưa cũng chẳng sao, ngồi sau xe người ấy lãng mạn như phim!");
  } else {
    console.log(
      "Cô đơn dưới mưa... Thôi đặt bát bún bò về nhà ăn cho ấm bụng.");
  }
} else {
  console.log("Trời khô ráo, đi đá bóng thôi anh em ơi!");
}
```

Điều kiện lồng nhau

Một số lưu ý

- Tránh lồng code quá sâu (Deep nesting)
- Lồng sâu làm giảm Readability

```
let isRaining = false;
let hasGymMembership = true;
let isFeelingWell = true;

if (isRaining === false) {
  if (hasGymMembership === true) {
    if (isFeelingWell === true) {
      // Logic chính bị lồng sâu (Deep nesting)
      console.log("Let's go to the gym and work out!");
    } else {
      console.log("Stay home and rest to recover.");
    }
  } else {
    console.log("You need to sign up for a membership first.");
  }
} else {
  console.log("It's raining, better exercise at home.");
}
```

Áp dụng clean code

```
let isRaining = false;
let hasGymMembership = true;
let isFeelingWell = true;

// 1. Check negative conditions first (Phủ định trước)
if (isRaining === true) {
  console.log("It's raining, better exercise at home.");
}
// 2. Next barrier
else if (hasGymMembership === false) {
  console.log("You need to sign up for a membership first.");
}
// 3. Final check
else if (isFeelingWell === false) {
  console.log("Stay home and rest to recover.");
}
// 4. Main logic (Logic chính nằm ở cuối, rất thoáng)
else {
  console.log("Let's go to the gym and work out!");
}
```

Cấu trúc switch case

Cú pháp:

```
switch(expression) {  
    case x:  
        // code block  
        break;  
    case y:  
        // code block  
        break;  
    default:  
        // code block }
```

Dựa trên một điều kiện, switch chọn một hoặc nhiều khối mã để thực thi

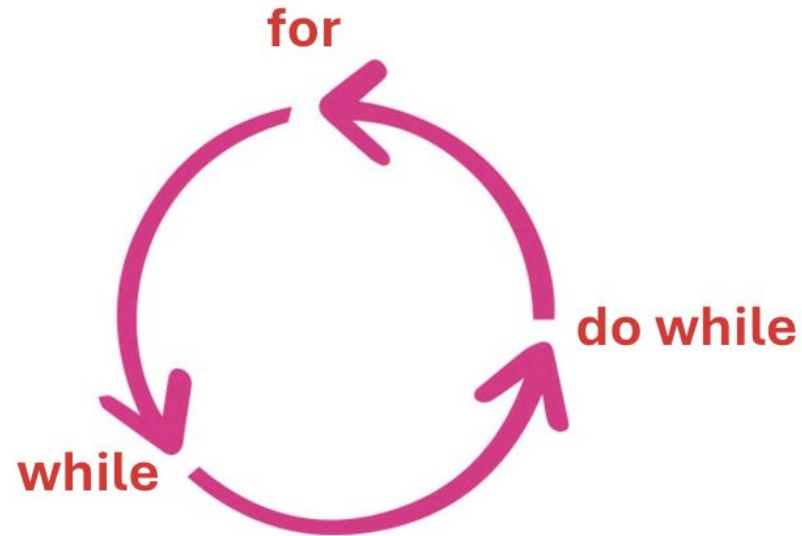
- switch thực thi các khối mã khớp với một biểu thức
- default: Khi không có case nào khớp
- switch thường được sử dụng như một giải pháp thay thế dễ đọc hơn cho nhiều câu lệnh if...else if...else, đặc biệt khi xử lý nhiều giá trị có thể.

switch hay if-else

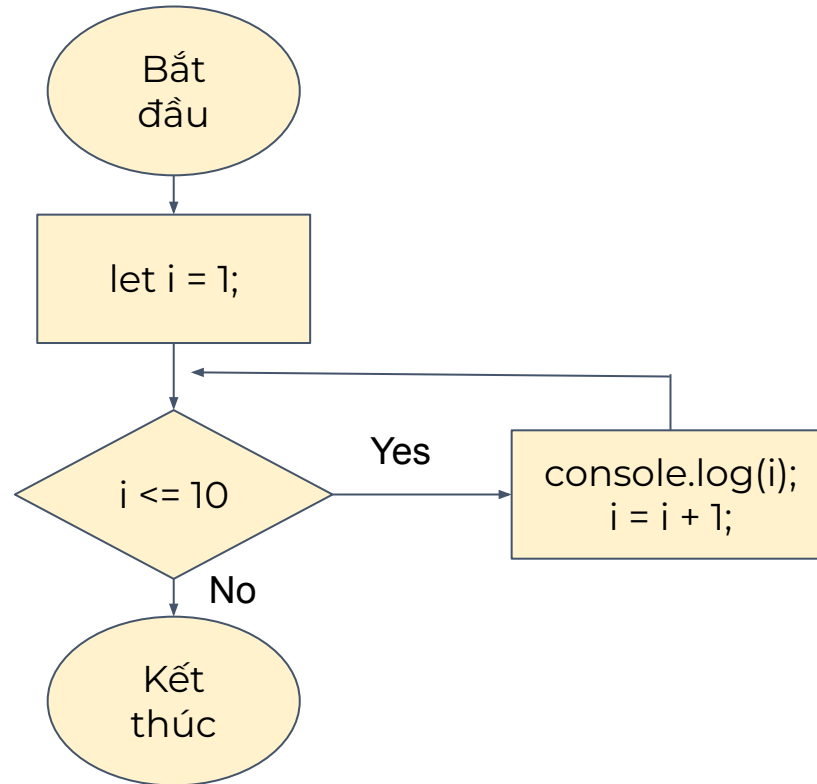
Tiêu chí	switch	if - else
Logic	Giá trị cụ thể	Biểu thức phức tạp
Cú pháp	Gọn gàng	Lồng nhau
Cách dùng	Dùng khi số lượng case cụ thể	Dùng cho logic so sánh phức tạp

2. Vòng lặp

Khái niệm: Vòng lặp rất hữu ích nếu bạn muốn chạy đi chạy lại cùng một mã, mỗi lần có một giá trị khác nhau.



Tìm hiểu sơ đồ vòng lặp

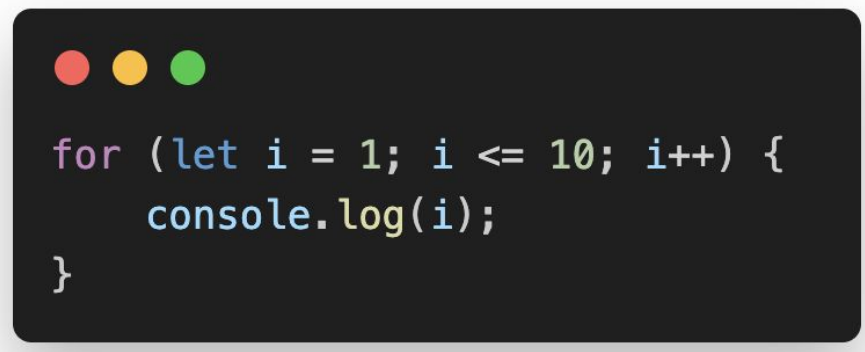


Vòng lặp for

Cú pháp:

```
for (initialization; condition; updation) {  
    // khối mã được thực thi  
};
```

- initialization: khởi tạo (chỉ thực hiện duy nhất lần đầu tiên)
- condition: điều kiện lặp
- updation: cập nhật giá trị (thực hiện sau cùng mỗi lần lặp)



```
for (let i = 1; i <= 10; i++) {  
    console.log(i);  
}
```

Vòng lặp while

Cú pháp:

```
while (condition) {  
    // khối mã được thực thi  
}
```

- Vòng lặp while thực thi một khối mã miễn là điều kiện được chỉ định là đúng



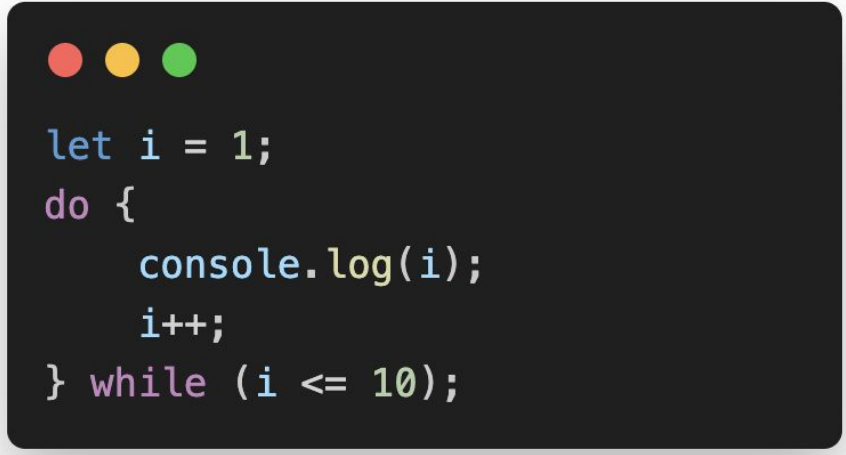
```
let i = 1;  
while (i <= 10) {  
    console.log(i);  
    i++;  
}
```

Vòng lặp do while

Cú pháp:

```
do {  
    // khối mã được thực thi  
} while (condition);
```

- Vòng lặp này sẽ thực thi khối mã một lần, trước khi kiểm tra xem điều kiện có đúng hay không, sau đó nó sẽ lặp lại vòng lặp miễn là điều kiện đó còn đúng.



```
let i = 1;  
do {  
    console.log(i);  
    i++;  
} while (i <= 10);
```

So sánh giữa for, while và do while

Đặc điểm	Vòng lặp for	Vòng lặp while	Vòng lặp do while
Số lần lặp	Thường dùng khi đã biết trước số lần lặp cụ thể	Thường dùng khi chưa biết rõ số lần lặp (phụ thuộc điều kiện)	Tương tự while, dùng khi chưa biết rõ số lần lặp
Kiểm tra điều kiện	Kiểm tra trước khi thực hiện thân vòng lặp	Kiểm tra trước khi thực hiện thân vòng lặp	Kiểm tra sau khi thực hiện thân vòng lặp ít nhất 1 lần
Số lần chạy tối thiểu	0 lần (nếu điều kiện sai ngay từ đầu)	0 lần (nếu điều kiện sai ngay từ đầu)	ít nhất 1 lần

Từ khóa break

Khái niệm:

- Dùng để kết thúc vòng lặp ngay lập tức
- Thoát hoàn toàn ra ngoài cấu trúc lặp hiện tại

```
for (let i = 1; i <= 10; i++) {  
  if (i === 3) {  
    break;  
  }  
  console.log(i);  
}
```

Từ khóa continue

Khái niệm:

- Dùng để bỏ qua lần lặp hiện tại
- Chuyển ngay sang lần lặp tiếp theo

```
for (let i = 1; i <= 10; i++) {  
    if (i === 3) {  
        continue;  
    }  
    console.log(i);  
}
```

Vòng lặp vô hạn

- Gây vòng lặp vô hạn
- Gây treo máy / ứng dụng

```
let count = 0;  
while (true) {  
  
    // Lỗi: Thiếu update hoặc break  
    console.log(count);  
}
```

- ❑ **Nắm vững cách sử dụng các câu lệnh có điều kiện (if, else if, else) và cấu trúc switch-case để thực hiện các hành động khác nhau tùy thuộc vào điều kiện**
- ❑ **Biết cách tránh lồng code quá sâu (Deep nesting) để tăng tính dễ đọc (Readability) cho mã nguồn**
- ❑ **Sử dụng thành thạo vòng lặp for, while, và do-while, cũng như phân biệt được sự khác biệt giữa chúng**
- ❑ **Biết cách sử dụng break để thoát khỏi vòng lặp ngay lập tức và continue để bỏ qua lần lặp hiện tại**
- ❑ **Hiểu được tác hại của vòng lặp vô hạn gây treo ứng dụng/máy tính**



HỌC VIỆN ĐÀO TẠO LẬP TRÌNH CHẤT LƯỢNG NHẬT BẢN